Relazione progetto finale SdC

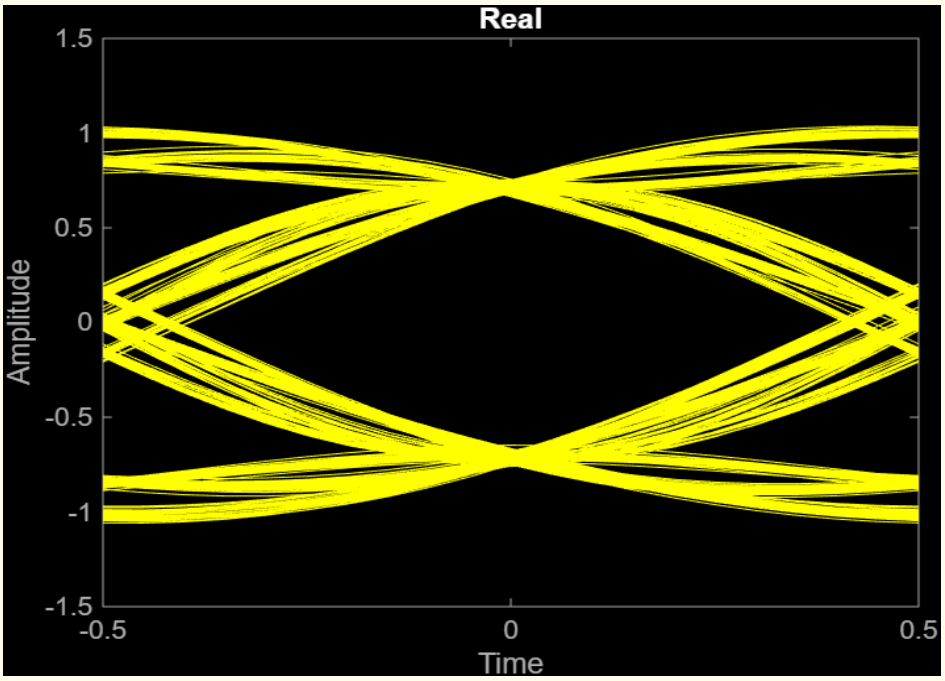
L'obiettivo è la trasmissione e la ricezione di un segnale testuale inserito da terminale, tramite l'utilizzo di due SDR-Pluto.

SIMULAZIONI

Abbiamo iniziato facendo delle simulazioni su Matlab senza l'utilizzo delle Pluto per comprendere gli effetti del canale e del rumore AWGN sul segnale e per provare diverse modulazioni.

La prima è servita per comprendere il funzionamento base della trasmissione e ricezione su un canale ideale, mentre la seconda ci ha dato un'idea migliore sugli effetti di una trasmissione in spazio libero utilizzando l'oggetto Matlab phasedFreeSpace che introduce i concetti di FSPL (free space path loss) e sfasamento dato dalla molteplicità non intera della lunghezza d'onda λ. Abbiamo simulato diverse distanze impostando le coordinate "fittizie" del ricevitore e del trasmettitore testando così diverse distanze tra trasmettitore e ricevitore, e diversi SNR.

Al termine di entrambe le simulazioni viene mostrato un diagramma ad occhio: in quella con canale ideale l'occhio è perfetto mentre in quella con rumore awgn l'immagine è più rappresentativa della realtà.
Riportiamo di seguito l'immagine del diagramma ad occhio della simulazione in spazio libero:



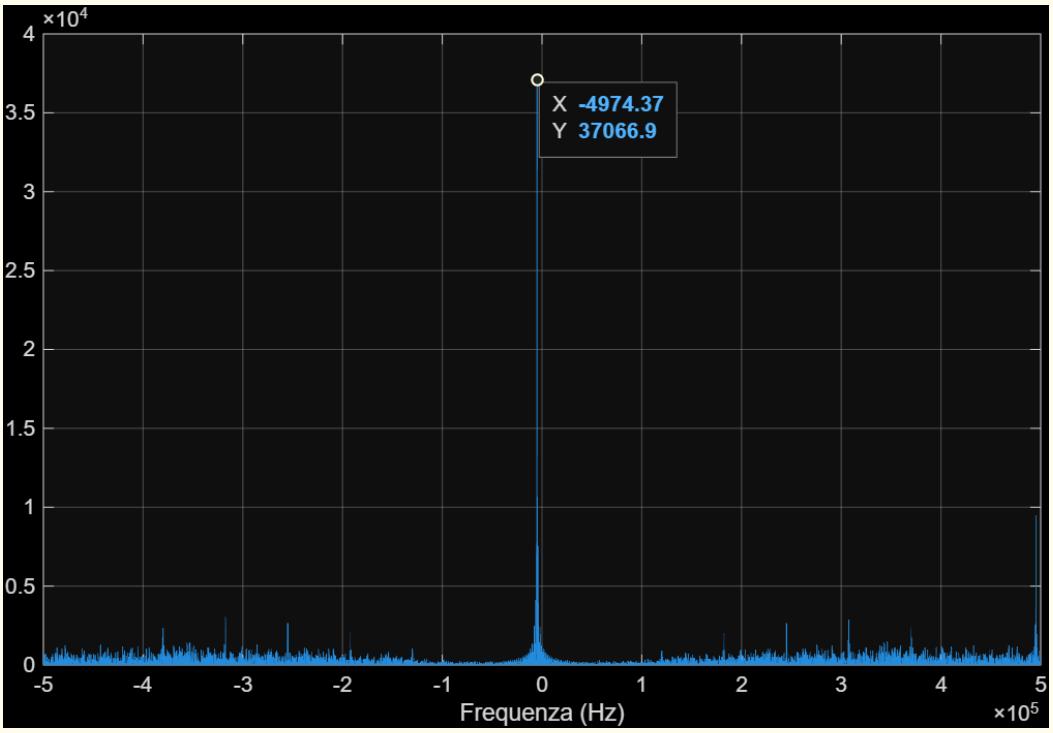
STESURA CODICI TX E RX

Abbiamo successivamente utilizzato le Pluto per trasmettere un testo fisso.
Al codice del trasmettitore abbiamo utilizzato una conversione da caratteri a bit tramite il codice ASCII per poi modularlo con QPSK e trasmetterlo con il comando transmitRepeat, attraverso un filtro RRC (root raised cosine) e un upsampling.
Poiché non vi è necessità né di alte velocità di trasmissione né di trasmissione di grandi quantità di dati siamo successivamente passati a una modulazione BPSK, in modo da rendere la trasmissione più robusta con regioni di decisione più ampie e quindi minore probabilità di errore.

Inizialmente venivano ricevuti solo dati casuali, in quanto nel codice del ricevitore non era stata implementata la correzione del CFO (carrier frequency offset) e il campionamento.
Per risolvere il problema del CFO abbiamo quindi utilizzato un algoritmo chiamato M-Th Power per una correzione più "grezza", che utilizza semplici passaggi matematici riguardanti l'elevamento ^M (con M numero simboli della modulazione utilizzata).
Matematicamente ciò che succede al segnale ricevuto r(t) è che vengono eliminate le informazioni relative alla modulazione contenute nella fase e poi viene eseguita la FFT per trovare la stima del CFO, cioè (con modulazione BPSK nel nostro caso):

1. $r(t)^2 = 1*e^{j*2pi*2*\Delta f*t}$
2. $FFT\{r(t)^2\}$

Ciò che si ottiene è quindi un impulso nel dominio della frequenza posizionato a 2*estimatedCFO, possiamo quindi procedere alla correzione moltiplicando per l'esponenziale cambiato di segno.
Di seguito riportiamo il picco dato dalla FFT che riporta la stima del CFO:



Abbiamo successivamente implementato l'oggetto Matlab carrierSynchronizer per una correzione più "fine", il quale utilizza un ciclo PLL (phase locked loop) per correggere lo sfasamento di ogni punto della costellazione calcolando l'angolo di differenza tra il punto ricevuto effettivamente e quello ideale della costellazione considerata.
Il funzionamento matematico "rigoroso" è il seguente:

Consideriamo sul piano complesso I/Q il punto corretto della modulazione BPSK e l'ipotetico punto ricevuto (x) che sarà sfasato di un certo angolo Φx.
Scriviamo Φx, angolo del punto ricevuto, e $\Phi 0$, angolo del simbolo corretto BPSK (cioè 0). Possiamo dunque scrivere i simboli rispettivamente come

- $Z = X+j\times Y$
- $S = Si + j\times Sq$

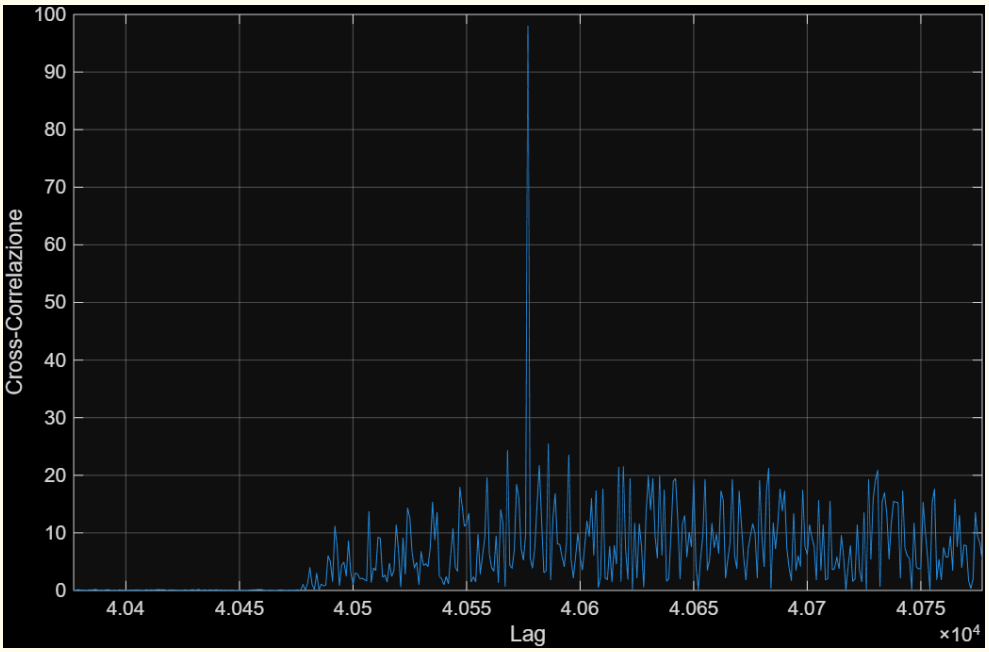
Possiamo calcolare $\theta = \Phi x - \Phi 0$ come

- $\arctan(Re[Z\times S^*] / Im[Z\times S^*])$, con $Z\times S^* = e^{j(\Phi x-\Phi 0)}$

per θ piccoli, $\theta \sim \sin(\theta) = Im[Z\times S^*] = Y\times Si - X\times Sq$
Concludo moltiplicando per $Ze^{-i\theta}$.
Questo procedimento viene iterato nel PLL per ogni punto ricevuto.

A questo punto ci siamo imbattuti nel problema di quando iniziare la ricezione, in quanto la funzione transmitRepeat trasmette continuamente e periodicamente l'input fornito e il ricevitore deve capire quando iniziare a catturare il segnale: la soluzione è introdurre sia nel trasmettitore che nel ricevitore una sequenza di bit nota (header) scritto inizialmente a mano con 26 bit, implementando successivamente un generatore pseudo-casuale di 100 bit con un seme (così che per tx e rx fosse lo stesso).

Inserita questa sequenza nota per capire dove iniziare a catturare al ricevitore, questo catturava un range piuttosto ampio di campioni che in seguito cross-correlavamo con l'header noto così da trovare il picco e capire quindi la posizione della sequenza per poter leggere il messaggio posizionato subito dopo di essa.
Riportiamo di seguito l'immagine del picco della cross-correlazione:



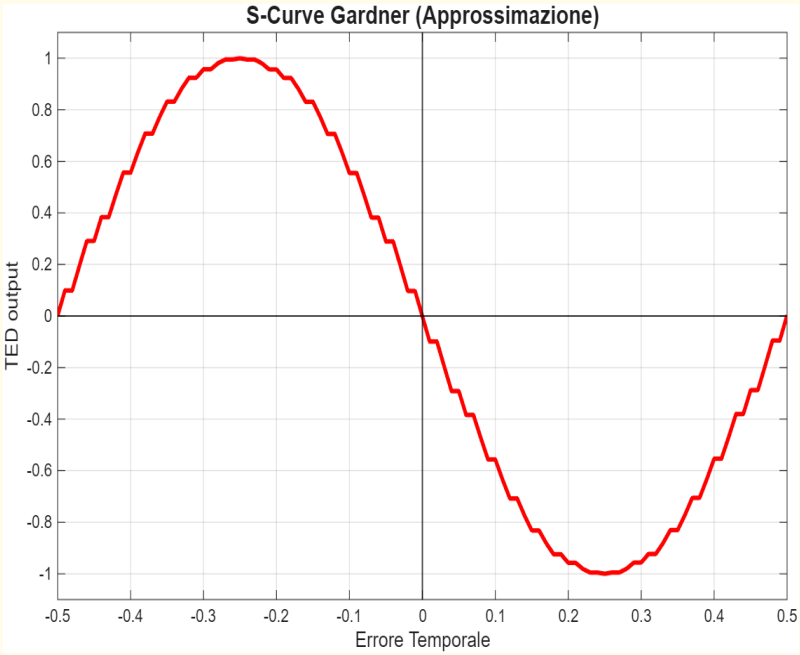
All'uscita del carrierSynchronizer, in alcuni casi, la costellazione poteva essere "ribaltata" e per correggere questo errore abbiamo moltiplicato il segnale ricevuto per la fase cambiata di segno del massimo della cross-correlazione.
Inoltre, Matlab restituiva talvolta un errore in quanto i bit catturati non erano un multiplo di 8 e quindi non convertibili tramite ASCII. Per ovviare a questo problema è stato inserito un piccolo script che aggiunge degli zeri per arrivare a un multiplo di 8 (script eliminato successivamente grazie ad altre migliorie al codice).

Nei test di ricezione talvolta veniva ricevuto il testo corretto intervallato da una serie di caratteri casuali, altre volte esclusivamente caratteri casuali. I problemi erano due: innanzitutto il campionamento non avveniva dinamicamente ma soltanto ogni sps (sample per symbol) campioni e questo non garantiva il campionamento nel punto di massimo; in secondo luogo vi era una problematica legata alla trasmissione continua che poteva potenzialmente congestionare il buffer della pluto in ricezione.

Per risolvere queste problematiche sono state utilizzate due strategie:

1. Introduzione di pause di trasmissione di 5ms composte da zeri tra ogni messaggio inviato
2. Utilizzo dell'oggetto Matlab symbolSynchronizer (anch'esso funzionante con un PLL) che, a seguito di un interpolatore parabolico il quale restituisce i massimi dei "set" di campioni considerati, viene posto il TED (timing error detector) che utilizza l'algoritmo gardner per stimare l'errore di campionamento (timing) tramite un'approssimazione della derivata del segnale ricevuto; matematicamente:
   - $e = (y[k-1] - y[k]) \times y^*[k-0.5]$, dove $y[k]$ è il "massimo stimato attuale", $y[k-1]$ il "massimo precedente", $y[k-0.5]$ il campione di mezzo tra i due

   Graficamente l'errore medio risulta essere circa quello riportato di seguito:



Il compito del TED è quello di “avvicinare” i campionamenti agli zeri della funzione, in base al segno di e, cioè:

- $e = 0 \rightarrow$ campionamento ottimo
- $e > 0 \rightarrow$ campionamento in anticipo
- $e < 0 \rightarrow$ campionamento in ritardo

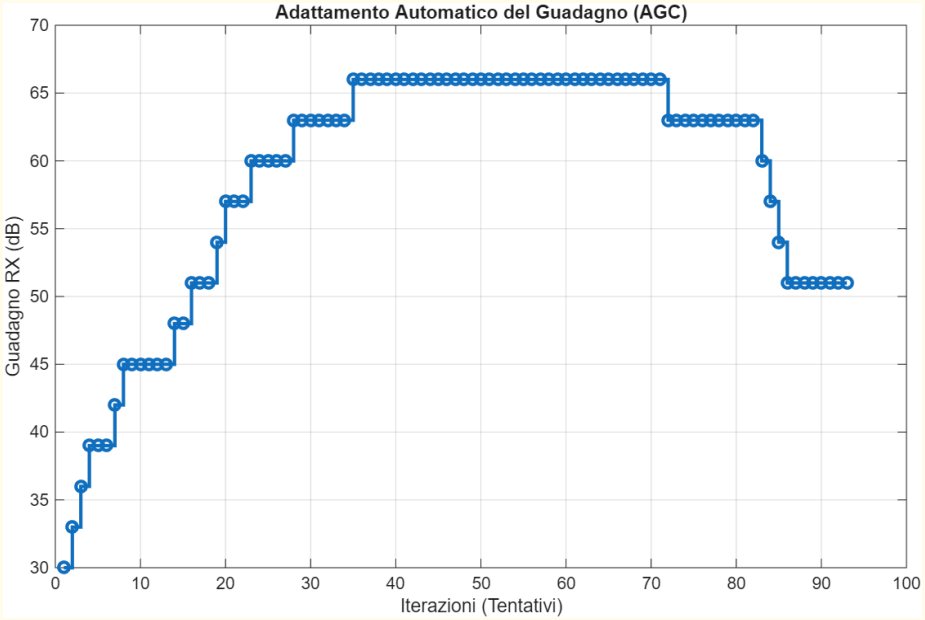
Il problema dei caratteri casuali contenuti nel messaggio altrimenti correttamente ricevuto non era ancora stato completamente risolto poiché il ricevitore tentava di catturare, demodulare e stampare le pause di zeri e il rumore che gli si sovrapponeva.

Per risolvere anche questo problema è stato implementato un piccolo script in trasmissione che calcola la lunghezza del testo e la codifica in binario scrivendola su 8 bit (limitando quindi la trasmissione a una stringa di massimo 255 caratteri), per poi trasmetterla insieme a header e messaggio nella forma [header - lunghezza - messaggio]. Il codice in ricezione estrapola la parte della lunghezza utilizzando la cross-correlazione di cui abbiamo parlato precedentemente e la decodifica per sapere quanti simboli elaborare per poi stampare il messaggio corretto privo di caratteri casuali a seguire.

Prima di mostrare il messaggio ricevuto abbiamo aggiunto una parte di codice che con la funzione biterr() confronta l’header ricevuto con quello noto e conta il numero di bit che non coincidono, rendendo così possibile impostare una soglia che determina se il messaggio ricevuto viene mostrato o meno.
Abbiamo attribuito a tale soglia un valore molto basso: 0 o 1 bit, in modo tale da stampare soltanto se l’header ricevuto non contiene errori.

Sono state successivamente implementate alcune accortezze finali al codice: innanzitutto una gestione automatica del gain del ricevitore, costituita da un ciclo while nel quale viene analizzato il picco del modulo del segnale ricevuto e, in base a questo, il gain viene aumentato o diminuito di 3dB (dai 30dB di partenza). Il ciclo si ripete finché il valore del gain è tale da far ricevere alla Pluto un picco compreso entro dei valori che abbiamo prestabilito in base a vari test di ricezione in diverse situazioni.
In secondo luogo vi è un controllo che impedisce il superamento del limite tecnico di gain della Pluto, evitando così errori dati da valori di gain fuori scala che abbiamo riscontrato diverse volte prima di implementare tale controllo.

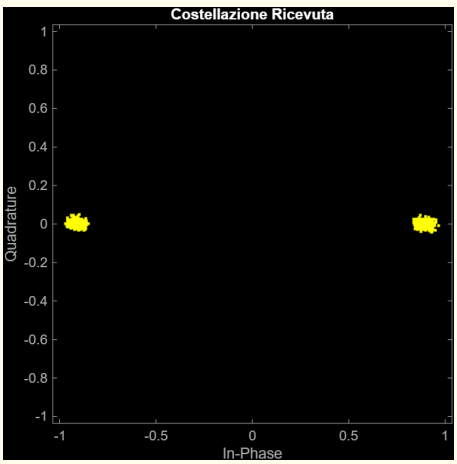
Di seguito riportiamo la variazione del gain basata sulle iterazioni del ciclo durante allontanamento e avvicinamento delle Pluto (arrivando a una distanza di circa 10m per poi tornare indietro) all’interno dell’edificio BL27 della sede di Bovisa del Politecnico di Milano:

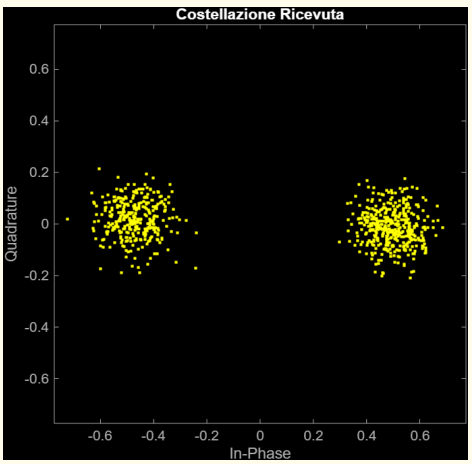


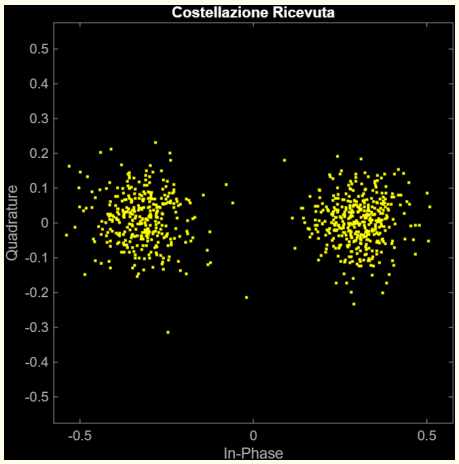
RISULTATI SPERIMENTALI

Una volta ultimato il codice finale del progetto abbiamo effettuato diversi test al fine di valutare la robustezza del codice in diverse condizioni.

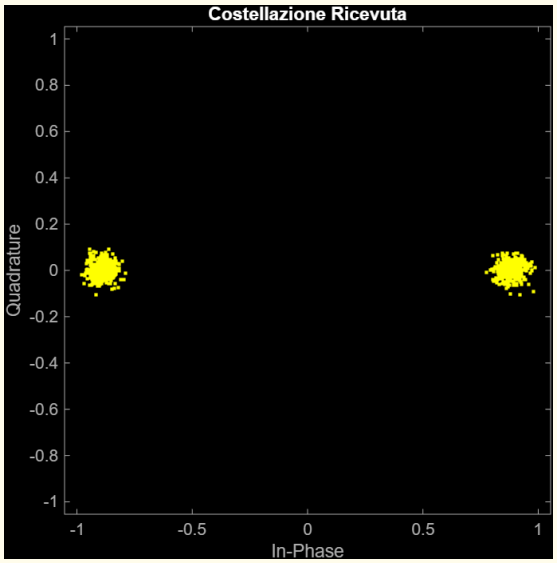
I primi test sono stati effettuati in ambiente rumoroso (edificio BL27 del Politecnico sede di Bovisa) a molteplici distanze, testando la trasmissione alle frequenze di 2.4 e 5.8 GHz. Nel primo caso i risultati sono stati positivi fino a una distanza di circa 10 metri, dove il gain veniva alzato al valore massimo e in caso di ulteriore allontanamento la ricezione risultava errata.
Di seguito riportiamo le costellazioni per il caso appena citato alle rispettive distanze di 1m, 5m circa e 10m circa:

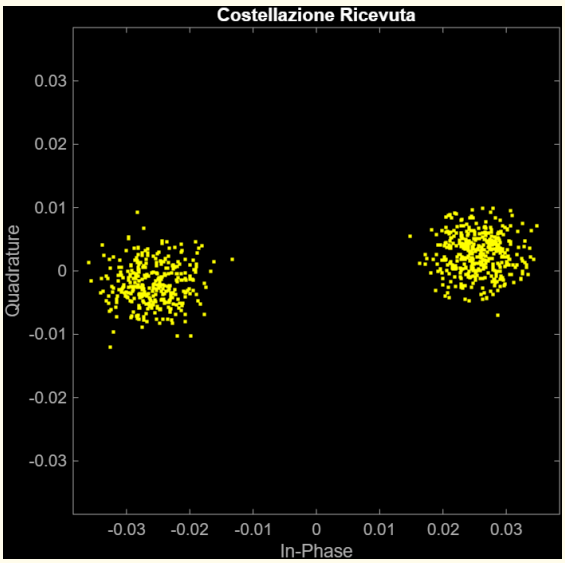




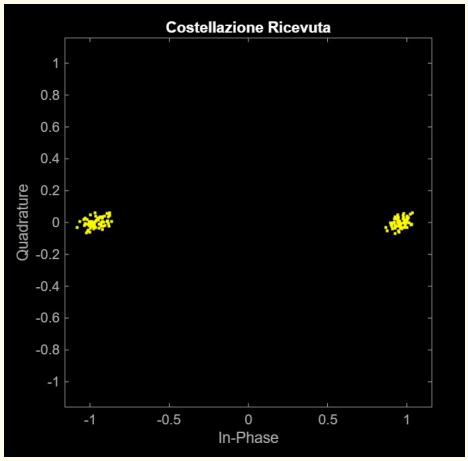


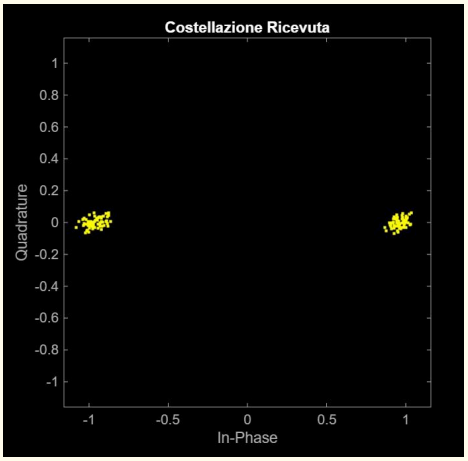
Nel secondo caso (5.8Ghz) i risultati sono stati peggiori: la trasmissione non riusciva a coprire una distanza superiore a 5 metri. Questo è in linea con le nostre aspettative in quanto all'aumentare della frequenza portante aumenta FSPL (free space path loss) secondo la relazione $FSPL = (4\times\pi\times d\times f/c)^2$.
Di seguito riportiamo le costellazioni alle rispettive distanze di 1m e circa 5m:

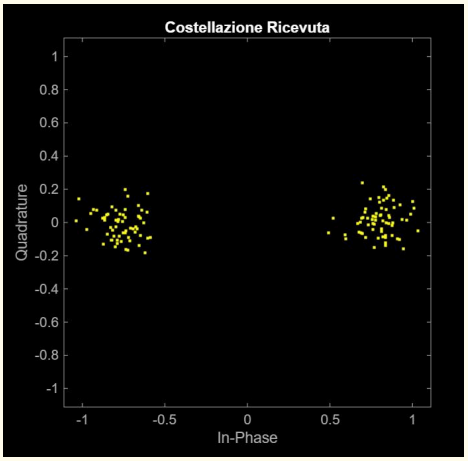


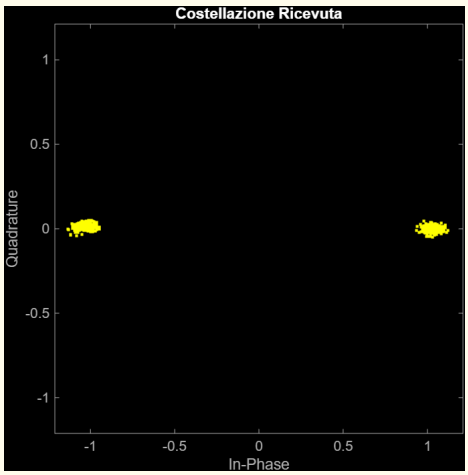


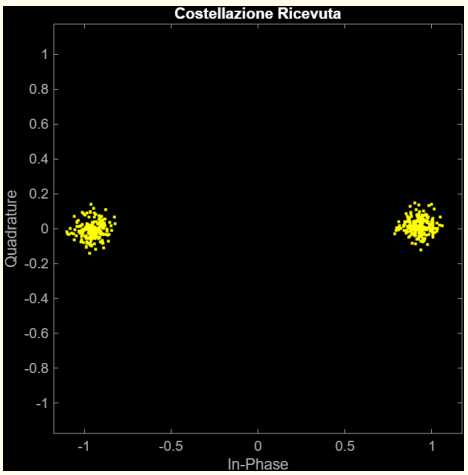
Gli stessi test sono poi stati effettuati sia in ambiente tranquillo (appartamento) con risultati chiaramente migliori, sia outdoor dove la ricezione è ulteriormente migliorata permettendoci di ricevere fino a 10 metri di distanza alla frequenza di 5.8 GHz.
Di seguito riportiamo le costellazioni dei test in ambiente tranquillo a 2.4 GHz alle rispettive distanze di 1m, 5m e 15m (circa) e outdoor a 2.4 GHz alle rispettive distanze di 1m, 5m circa, 10m (circa):

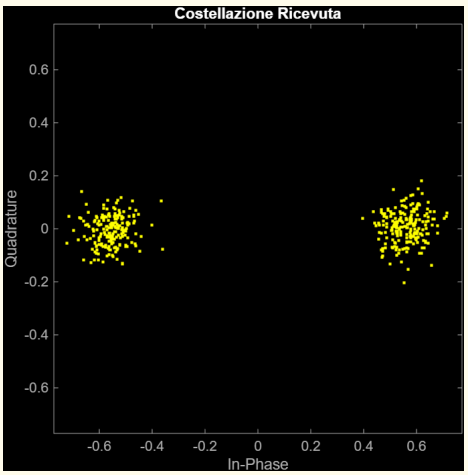
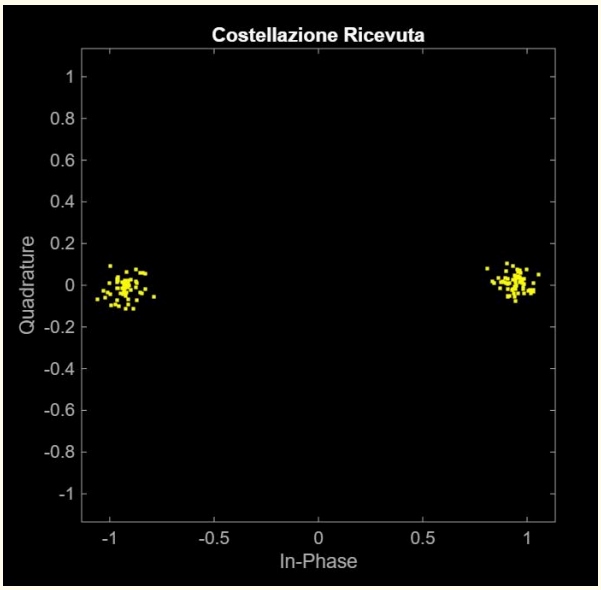
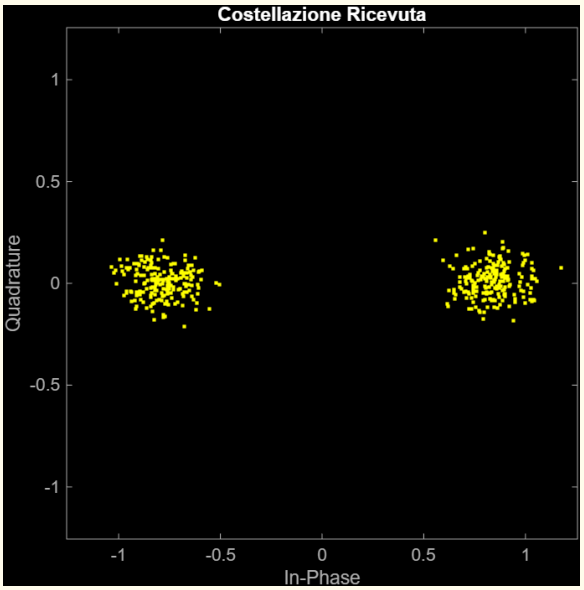
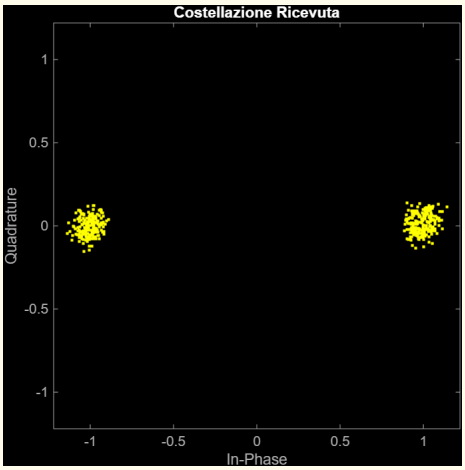
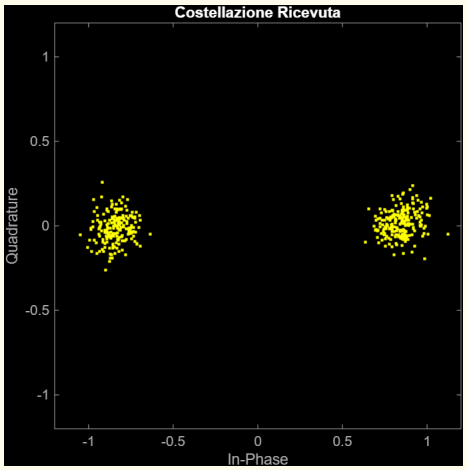
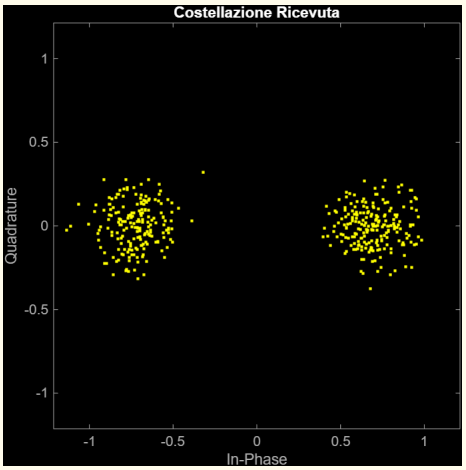












In conclusione riportiamo le costellazioni ricevute nei test in ambiente tranquillo (alle rispettive distanze di 1m e 5m circa) e outdoor (alle rispettive distanze di 1m, 5m circa, 10m circa) a 5.8GHz: